Assignment 3: MARS Fundamentals

Name: Hiram Mora Sauceda

RedID: 132765669

Course / Section: CS 240

GitHub repository: https://github.com/hmorasauceda2166-boop/Assignment-3-MARS-Fundamentals

All five programs were assembled and run in MARS 4.5. They are in the repository as `hello_world.asm`, `print_1_to_100.asm`, `sum_even.asm`, `memory_add.asm` and `fizzbuzz.asm`. Each section below has the full source code, the output, and the register trace.

Program 1 - Hello World

Prints Hello, World! with syscall 4 and exits with syscall 10.

Source code (hello_world.asm)

```
# Assignment 3 - MARS Fundamentals
# Program 1: Hello World
# Name: Hiram Mora Sauceda   RedID: 132765669
# Prints "Hello, World!" using syscall 4 (print string), then exits.

        .data
message:    .asciiz "Hello, World!\n"

        .text
main:
        li   $v0, 4              # syscall 4 = print string
        la   $a0, message        # $a0 = address of the string
        syscall

done:
        li   $v0, 10             # syscall 10 = exit
        syscall
```

Output

```
Hello, World!
```

Register trace (final state after the program exits, as shown in the MARS Registers pane)

Register	Decimal	Hex	Why
$v0	10	0x0000000A	syscall code for exit (10)
$a0	268500992	0x10010000	address of the message string in the data segment
$at	268500992	0x10010000	assembler temporary used by the la pseudo-instruction

Program 2 - Print 1 through 100

A counter in $t0 starts at 1. Each pass prints it and adds 1, and the loop stops when the counter is greater than 100.

Source code (print_1_to_100.asm)

```
# Assignment 3 - MARS Fundamentals
# Program 2: Print the integers 1 through 100
# Name: Hiram Mora Sauceda   RedID: 132765669
# Counts from 1 to 100 with a loop and prints each number on its own line.

        .data
newline:    .asciiz "\n"

        .text
main:
        li   $t0, 1              # $t0 = counter, starts at 1
        li   $t1, 100            # $t1 = last number to print

loop:
        bgt  $t0, $t1, done      # if counter > 100, stop

        li   $v0, 1              # syscall 1 = print integer
        move $a0, $t0            # number to print
        syscall

        li   $v0, 4              # syscall 4 = print string
        la   $a0, newline
        syscall

        addi $t0, $t0, 1         # counter++
        j    loop

done:
        li   $v0, 10             # exit
        syscall
```

Output (the program prints one number per line; shown 10 per row here to save space)

```
  1   2   3   4   5   6   7   8   9  10
 11  12  13  14  15  16  17  18  19  20
 21  22  23  24  25  26  27  28  29  30
 31  32  33  34  35  36  37  38  39  40
 41  42  43  44  45  46  47  48  49  50
 51  52  53  54  55  56  57  58  59  60
 61  62  63  64  65  66  67  68  69  70
 71  72  73  74  75  76  77  78  79  80
 81  82  83  84  85  86  87  88  89  90
 91  92  93  94  95  96  97  98  99 100
```

Register trace (final state after the program exits)

Register	Decimal	Hex	Why
$t0	101	0x00000065	counter went one past 100, which is why the loop stopped
$t1	100	0x00000064	last number to print
$v0	10	0x0000000A	syscall code for exit (10)
$a0	268500992	0x10010000	address of the newline string
$at	1	0x00000001	result of the last slt inside bgt (1 = counter was greater than the limit)

Tests (changing the 100 in `li $t1, 100`)

Last number	Lines printed	First line	Last line
100	100	1	100
1	1	1	1
0	0	(nothing)	(nothing)

Program 3 - Sum of the even integers 1 through 100

Starts at 2 and adds every even number up to 100 (2 + 4 + ... + 100 = 2550).

Source code (sum_even.asm)

```
# Assignment 3 - MARS Fundamentals
# Program 3: Sum of the even integers from 1 through 100
# Name: Hiram Mora Sauceda   RedID: 132765669
# Starts at 2 and adds every even number up to 100.
# Expected answer: 2 + 4 + ... + 100 = 2550

        .data
label:      .asciiz "Sum of the even integers from 1 through 100: "
newline:    .asciiz "\n"

        .text
main:
        li   $t0, 2             # $t0 = current even number (first one is 2)
        li   $t1, 100           # $t1 = limit
        li   $t2, 0             # $t2 = running sum

loop:
        bgt  $t0, $t1, done     # past the limit, stop
        add  $t2, $t2, $t0      # sum += current
        addi $t0, $t0, 2        # next even number
        j    loop

done:
        li   $v0, 4             # print the label
        la   $a0, label
        syscall

        li   $v0, 1             # print the sum
        move $a0, $t2
        syscall

        li   $v0, 4             # newline
        la   $a0, newline
        syscall

        li   $v0, 10            # exit
        syscall
```

Output

```
Sum of the even integers from 1 through 100: 2550
```

Register trace (final state after the program exits)

Register	Decimal	Hex	Why
$t0	102	0x00000066	next even number that would have been added (102 > 100)
$t1	100	0x00000064	limit
$t2	2550	0x000009F6	final sum
$a0	268501038	0x1001002E	address of the newline string
$v0	10	0x0000000A	syscall code for exit (10)
$at	268500992	0x10010000	assembler temporary used by la

Loop trace ($t0 and $t2 when the loop reaches the check again after n additions)

n (additions done)	$t0	$t2 (running sum)
1	4	2
2	6	6
3	8	12
4	10	20
5	12	30
10	22	110
25	52	650
49	100	2450
50	102	2550

Tests (changing the 100 in `li $t1, 100`)

Limit	Printed sum	Expected
100	2550	2550
99	2450	2450
2	2	2
1	0	0
0	0	0

Program 4 - Load, add, and store in memory

Loads two words from memory into registers, adds them, stores the sum back to memory, then loads it again and prints it to confirm.

Source code (memory_add.asm)

```
# Assignment 3 - MARS Fundamentals
# Program 4: Load two integers from memory, add them, store the result back
# Name: Hiram Mora Sauceda   RedID: 132765669
# num1 and num2 are loaded into registers, added, and the sum is stored
# in the "result" word. The result is then loaded again and printed to
# confirm it really was written to memory.

        .data
num1:       .word 25
num2:       .word 17
result:     .word 0
msg:        .asciiz "Result stored in memory: "
newline:    .asciiz "\n"

        .text
main:
        lw   $t0, num1           # $t0 = num1
        lw   $t1, num2           # $t1 = num2
        add  $t2, $t0, $t1       # $t2 = num1 + num2
        sw   $t2, result         # store the sum back to memory

        lw   $t3, result         # load it back to check

        li   $v0, 4              # print label
        la   $a0, msg
        syscall

        li   $v0, 1              # print the value we read from memory
        move $a0, $t3
        syscall

        li   $v0, 4              # newline
        la   $a0, newline
        syscall

done:
        li   $v0, 10             # exit
        syscall
```

Output

```
Result stored in memory: 42
```

Memory trace (data segment)

Address	Label	Before run	After run
0x10010000	num1	25	25
0x10010004	num2	17	17
0x10010008	result	0	42

Register trace (final state after the program exits)

Register	Decimal	Hex	Why
$t0	25	0x00000019	num1 loaded with lw
$t1	17	0x00000011	num2 loaded with lw
$t2	42	0x0000002A	num1 + num2
$t3	42	0x0000002A	result loaded back from memory after sw
$v0	10	0x0000000A	syscall code for exit (10)
$a0	268501030	0x10010026	address of the newline string
$at	268500992	0x10010000	assembler temporary used to build data addresses for lw/sw/la

Tests (changing the .word values in the data segment)

num1	num2	Result
25	17	42
-5	5	0
-10	-20	-30
0	0	0
2147483647	0	2147483647
2147483647	1	MARS stops with: Runtime exception at 0x00400010: arithmetic overflow (add traps on signed overflow, addu would wrap)

Program 5 - FizzBuzz

Multiples of 15 print FizzBuzz, multiples of 3 print Fizz, multiples of 5 print Buzz, everything else prints the number. The 15 check goes first.

Source code (fizzbuzz.asm)

```
# Assignment 3 - MARS Fundamentals
# Program 5: FizzBuzz (1 through 100)
# Name: Hiram Mora Sauceda   RedID: 132765669
# Multiple of 15 -> FizzBuzz, multiple of 3 -> Fizz,
# multiple of 5 -> Buzz, otherwise print the number.
# The 15 check has to come first or FizzBuzz would never print.

        .data
fizz:       .asciiz "Fizz"
buzz:       .asciiz "Buzz"
newline:    .asciiz "\n"

        .text
main:
        li   $t0, 1              # $t0 = current number
        li   $t1, 100            # $t1 = last number
        li   $t5, 3
        li   $t6, 5
        li   $t7, 15

loop:
        bgt  $t0, $t1, done

        div  $t0, $t7            # number / 15, remainder goes to HI
        mfhi $t2
        beq  $t2, $zero, print_fizzbuzz

        div  $t0, $t5            # number / 3
        mfhi $t2
        beq  $t2, $zero, print_fizz

        div  $t0, $t6            # number / 5
        mfhi $t2
        beq  $t2, $zero, print_buzz

        li   $v0, 1              # not a multiple of 3 or 5, print the number
        move $a0, $t0
        syscall
        j    next

print_fizzbuzz:
        li   $v0, 4
        la   $a0, fizz
        syscall
        li   $v0, 4
        la   $a0, buzz
        syscall
        j    next

print_fizz:
        li   $v0, 4
        la   $a0, fizz
        syscall
        j    next

print_buzz:
        li   $v0, 4
        la   $a0, buzz
        syscall

next:
        li   $v0, 4              # newline after every line
        la   $a0, newline
        syscall

        addi $t0, $t0, 1
        j    loop

done:
        li   $v0, 10             # exit
        syscall
```

Output (one result per line in the console; line number added here)

```
  1: 1           21: Fizz        41: 41          61: 61          81: Fizz
  2: 2           22: 22          42: Fizz        62: 62          82: 82
  3: Fizz        23: 23          43: 43          63: Fizz        83: 83
  4: 4           24: Fizz        44: 44          64: 64          84: Fizz
  5: Buzz        25: Buzz        45: FizzBuzz    65: Buzz        85: Buzz
  6: Fizz        26: 26          46: 46          66: Fizz        86: 86
  7: 7           27: Fizz        47: 47          67: 67          87: Fizz
  8: 8           28: 28          48: Fizz        68: 68          88: 88
  9: Fizz        29: 29          49: 49          69: Fizz        89: 89
 10: Buzz        30: FizzBuzz    50: Buzz        70: Buzz        90: FizzBuzz
 11: 11          31: 31          51: Fizz        71: 71          91: 91
 12: Fizz        32: 32          52: 52          72: Fizz        92: 92
 13: 13          33: Fizz        53: 53          73: 73          93: Fizz
 14: 14          34: 34          54: Fizz        74: 74          94: 94
 15: FizzBuzz    35: Buzz        55: Buzz        75: FizzBuzz    95: Buzz
 16: 16          36: Fizz        56: 56          76: 76          96: Fizz
 17: 17          37: 37          57: Fizz        77: 77          97: 97
 18: Fizz        38: 38          58: 58          78: Fizz        98: 98
 19: 19          39: Fizz        59: 59          79: 79          99: Fizz
 20: Buzz        40: Buzz        60: FizzBuzz    80: Buzz       100: Buzz
```

Checked against the expected FizzBuzz sequence for 1 to 100: all 100 lines match.

Register trace (final state after the program exits)

Register	Decimal	Hex	Why
$t0	101	0x00000065	counter, one past 100
$t1	100	0x00000064	last number
$t2	0	0x00000000	remainder from the last check (100 mod 5)
$t5	3	0x00000003	divisor 3
$t6	5	0x00000005	divisor 5
$t7	15	0x0000000F	divisor 15
HI	0	0x00000000	remainder from the last div
LO	20	0x00000014	quotient from the last div (100 / 5)

Branch trace (last remainder checked when stopping at n)

n	$t2	HI (remainder)	LO (quotient)	Printed	Why
1	1	1	0	1	not a multiple of 15, 3 or 5, so the number prints
3	0	0	1	Fizz	remainder 0 on the divide by 3
5	0	0	1	Buzz	remainder 0 on the divide by 5
15	0	0	1	FizzBuzz	remainder 0 on the first check (divide by 15)

Tests (normal and edge cases from the full output)

Number	Printed	Expected
1	1	1
2	2	2
3	Fizz	Fizz
5	Buzz	Buzz
9	Fizz	Fizz
10	Buzz	Buzz
15	FizzBuzz	FizzBuzz
30	FizzBuzz	FizzBuzz
45	FizzBuzz	FizzBuzz
90	FizzBuzz	FizzBuzz
98	98	98
99	Fizz	Fizz
100	Buzz	Buzz

Instructions vs. pseudo-instructions

These are the pseudo-instructions my programs use and the real instructions MARS turns them into.

Pseudo-instruction	Used in	Real instruction(s) it assembles into
li $t0, 1	all programs	addiu $t0, $zero, 1
la $a0, message	all programs	lui $at, 0x1001 then ori $a0, $at, offset
move $a0, $t0	print programs	addu $a0, $zero, $t0
bgt $t0, $t1, done	loops	slt $at, $t1, $t0 then bne $at, $zero, done
lw $t0, num1 / sw $t2, result	memory_add	lui $at, 0x1001 then lw/sw with an offset from $at

Real instructions used directly
add, addi, j, beq, div, mfhi, syscall

Reflection

This assignment got me comfortable with MARS and with working at the register level. MIPS has no for loop, so every loop here is a counter register, a branch that exits, and a jump back to the top. Watching the registers made it easy to see where each value was, like $t2 building up to 2550 in the even sum and the remainder showing up in HI after each div in FizzBuzz. I also learned that li, la, move and bgt are pseudo-instructions that turn into real ones, and that lw and sw are the only way to move data between memory and registers. FizzBuzz showed me that order matters, because checking 3 or 5 first would never print FizzBuzz, so 15 has to be checked first. The edge cases I tested were a limit of 0 or 1, negative numbers, and an overflow, which makes add trap while addu would just wrap.

Sources

AI use: I used Claude to help write and test the .asm programs and to organize the output into this PDF. ZyBooks Chapter 3 was the reading for this assignment. I did not work with other students.